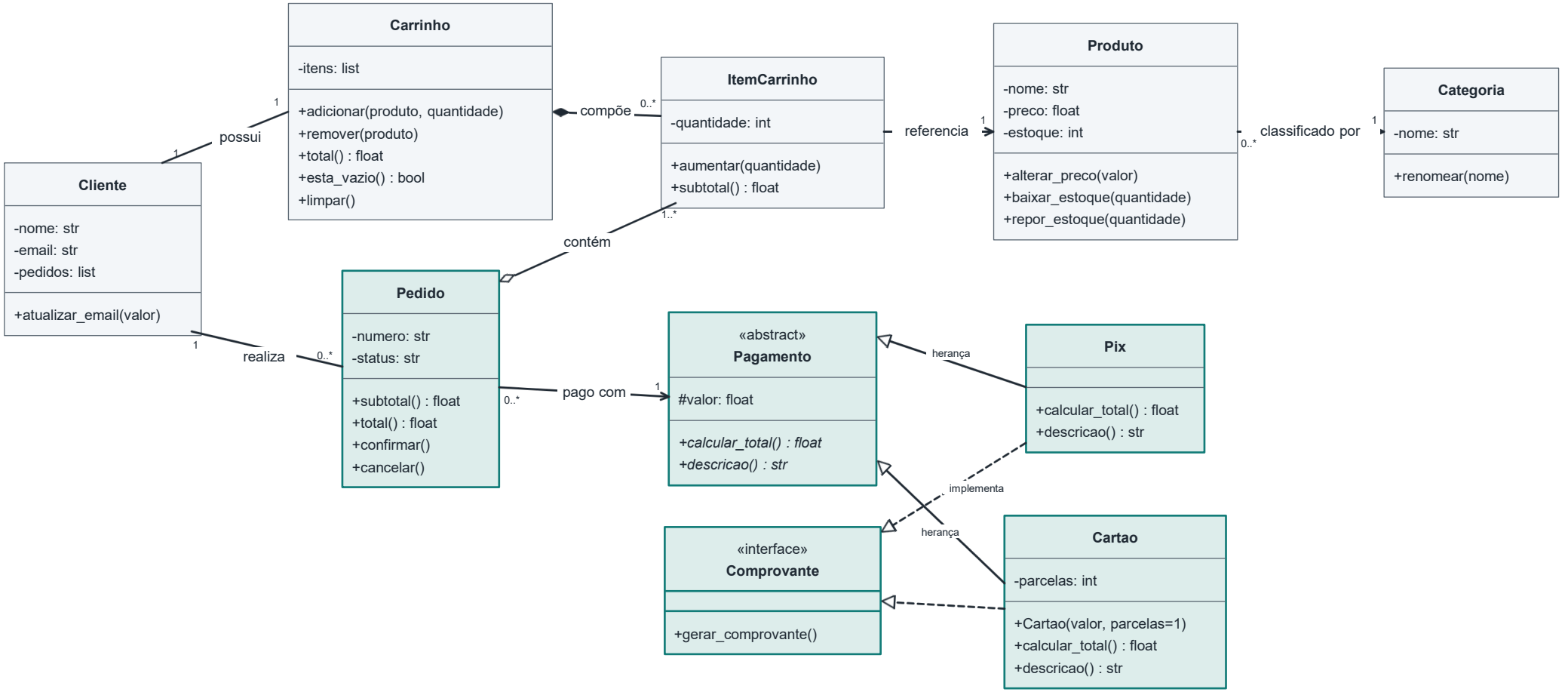
Pagamento, Comprovante, Pix, Cartao e Pedido



Em destaque: Pedido, Pagamento, Pix, Cartao, Comprovante

O que vamos construir hoje

- Revisão do `main.py`
- Diagrama de sequência resumido
- Tipos de herança: simples e múltipla
- Herança no quadro: Pix é um Pagamento

- Classe abstrata `Pagamento`
- Interface `Comprovante`
- Classe `Pix`: desconto de 5%
- Classe `Cartao`: `super()` e parcelas

- Intervalo: 20h20 às 20h40
- Sobreposição × sobrecarga
- Polimorfismo: lista com Pix e Cartao

- Classe `Pedido`: agregação
- `confirmar()` e `cancelar()`
- Orientações da atividade de 14/11

Regras de hoje: RN8 (Pix), RN9 e RN10 (Cartão), RN11 (estoque do pedido), RN6 e RN7 (carrinho).

Visão geral do fluxo

Cliente escolhe a forma de pagamento

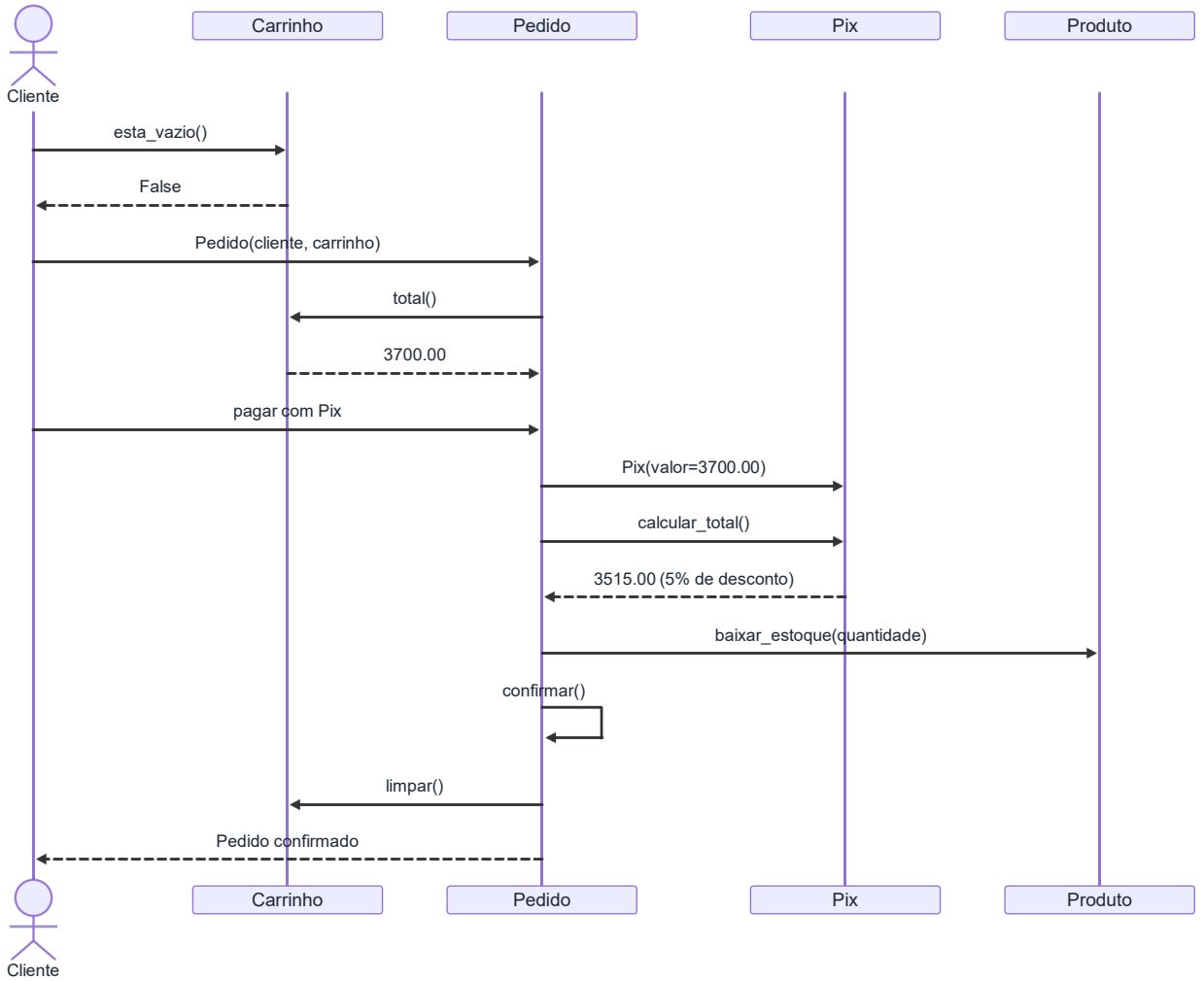
Pedido cria Pix ou Cartao

Pedido calcula o total

Pedido confirma e limpa o carrinho

Versão completa com o diagrama de sequência em 17/11

Finalização do pedido



Como ler o diagrama de sequência

Os elementos

- **Boneco:** o ator, quem usa o sistema.
- **Caixas:** os objetos que participam.
- **Linha vertical:** a vida do objeto no tempo.
- **Seta cheia:** uma chamada de método.
- **Seta tracejada:** a resposta.

A leitura

- Lê-se de **cima para baixo**: o que está em cima acontece antes.
- Cada seta sai de quem **chama** e chega em quem **executa**.
- Seta que volta para a mesma caixa: o objeto chama um método dele mesmo.

As regras no diagrama

- **RN6:** `esta_vazio()` responde False.
- **RN8:** `calcular_total()` devolve 3515.00.
- **RN11:** `baixar_estoque(quantidade)`.
- **RN7:** `limpar()` esvazia o carrinho.

Fonte: GUEDES, UML 2: uma abordagem prática · docs/diagrama-sequencia.md

Herança: uma classe recebe o que a outra já tem

Herança é quando uma classe filha recebe os atributos e os métodos de uma classe mãe, e pode acrescentar ou mudar o que precisar. A relação é **"é um"**: Pix **é um** Pagamento.

classe Pagamento:

```python
class Pagamento(ABC):
    def __init__(self, valor):
        if valor <= 0:
            raise ValueError("O valor do pagamento precisa ser maior que zero.")
        self._valor = float(valor)
```

class Pix(Pagamento): # Pix herda de Pagamento

```python
from models.pagamento import Pagamento


class Pix(Pagamento, Comprovante):
    DESCONTO = 0.05

    def calcular_total(self):
        return round(self._valor * (1 - Pix.DESCONTO), 2)
```

Main.py

Main

```python
print("--- Pagamento ---")

pix = Pix(3700)
cartao_3x = Cartao(3700, 3)
cartao_6x = Cartao(3700, 6)

print("RN8 - Pix (esperado 3515.0):", pix.calcular_total())
print("RN9 - Cartão em 3x (esperado 3700.0):", cartao_3x.calcular_total())
print("RN9 - Cartão em 6x (esperado 4070.0):", cartao_6x.calcular_total())

pagamentos = [pix, cartao_3x, cartao_6x]
for pagamento in pagamentos:
    print(pagamento.gerar_comprovante())

# Cartao(3700, 13)
```

- **Classe mãe** (base ou superclasse): `Pagamento`.
- **Classe filha** (subclasse): `Pix`.
- Sintaxe: `class Filha(Mae):`
- A filha usa `mostrar()` sem escrever o método de novo.
- Teste do "é um": Pix é um Pagamento? Se sim, a herança faz sentido.

Herança simples e herança múltipla

2 tipos de herança

Herança simples

A classe filha tem **uma** classe mãe.

```
class Cachorro(Animal):
```

É a forma mais comum e a mais fácil de entender.

Herança múltipla

A classe filha tem **mais de uma** classe mãe.

```
class Pato(Animal, Nadador, Voador):
```

No projeto: `class Pix(Pagamento, Comprovante):`

Vantagens

Reuso: o código da mãe vale para todas as filhas.

Manutenção: corrigir na mãe corrige em todas.

Organização: o código segue a relação “é um”.

Desvantagens

Acoplamento: mudar a mãe afeta todas as filhas.

Muitos níveis de herança deixam o código difícil de ler.

Na múltipla, métodos com o mesmo nome dependem da ordem das mães.

Fonte: PYTHON SOFTWARE FOUNDATION, O tutorial Python, seção Herança múltipla · exemplos/heranca_multipla.py

Pagamento: um modelo que não cria objetos

Classe abstrata é um modelo incompleto: ela define o que as classes filhas precisam ter, mas não cria objetos. Só as filhas (Pix e Cartao) criam.

```python
from abc import ABC, abstractmethod

class Pagamento(ABC):
    def __init__(self, valor):
        if valor <= 0:
            raise ValueError("O valor do pagamento precisa ser maior que zero.")
        self._valor = float(valor)

    @property
    def valor(self):
        return self._valor

    @abstractmethod
    def calcular_total(self):
        pass

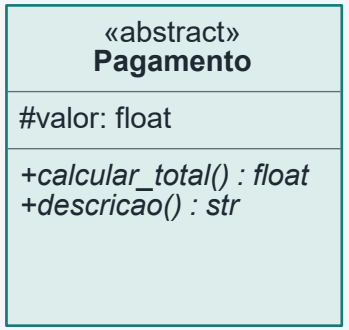
    @abstractmethod
    def descricao(self):
        pass
```

- `ABC` vem do módulo `abc` (Abstract Base Class) do python.
- `@abstractmethod` marca o método que toda filha é **obrigada** a escrever.
- `Pagamento(100)` gera `TypeError`.
- Pode ter código pronto: o `__init__` vale para Pix e Cartao.
- No diagrama: «abstract».

Fonte: PYTHON SOFTWARE FOUNDATION, abc: classes base abstratas · src/models/pagamento.py

Atributos em herança: Público, protegido e privado

A classe Pagamento tem atributo #Protegido para que outras classes filhas (que herdam) tenham acesso



Protegido · _valor

Um sublinhado na frente.

A classe e as classes **filhas** usam.

É um combinado entre programadores: o Python não bloqueia o acesso.

No projeto: `self._valor` em Pagamento.

+ Público · nome

Sem sublinhado na frente.

Qualquer parte do programa lê e altera.

No projeto: os métodos, como `total()` e `adicionar()`.

- Privado · __preco

Dois sublinhados na frente.

Só a **própria classe** usa.

O Python troca o nome por dentro, e o acesso de fora falha.

No projeto: `self.__preco` em Produto.

Por que protegido? Pix e Cartao precisam ler o valor para calcular o total. Com `__valor` (privado), as filhas não conseguiriam acessar.

Fonte: PYTHON SOFTWARE FOUNDATION, O tutorial Python, seção Variáveis privadas

Comprovante: um contrato para as classes

Interface é um contrato: ela só diz quais métodos a classe precisa ter, sem atributos e sem código pronto. Quem assina o contrato **tem a capacidade de** fazer aquilo.

Classe comprovante:

```python
from abc import ABC, abstractmethod


class Comprovante(ABC):
    @abstractmethod
    def gerar_comprovante(self):
        pass
```

Classe Pix

```python
from models.comprovante import Comprovante
from models.pagamento import Pagamento


class Pix(Pagamento, Comprovante):
    def gerar_comprovante(self):
        return f"Comprovante Pix: R$ {self.calcular_total():.2f} pagos à vista."
```

Pontos-chave

- O Python não tem a palavra `interface`: usamos `ABC` só com métodos abstratos.
- Pix e Cartao assinam o contrato: precisam escrever `gerar_comprovante()`.
- Se faltar o método, o objeto não é criado (`TypeError`).
- No diagrama: «`interface`» e seta tracejada.

Fonte: PYTHON SOFTWARE FOUNDATION, abc: classes base abstratas · src/models/comprovante.py

Quando usar cada uma

Classe abstrata · Pagamento

Pode ter **atributos** (`_valor`) e **métodos prontos** (`__init__`).

Relação **“é um”**: Pix é um Pagamento.

Use quando as classes compartilham dados e código.

No diagrama: triângulo com linha cheia.

Interface · Comprovante

Só define o **contrato**: `gerar_comprovante()`.

Relação **“tem a capacidade de”**: Pix gera comprovante.

Use quando classes diferentes precisam da mesma capacidade.

No diagrama: triângulo com linha tracejada.

No projeto: `class Pix(Pagamento, Comprovante)` herda da classe abstrata e implementa a interface ao mesmo tempo.

Fonte: KEOGH; GIANNINI, OOP desmistificado · GUEDES, UML 2: uma abordagem prática

Pix: 5% de desconto (RN8)

`class Pix`

```python
from models.comprovante import Comprovante
from models.pagamento import Pagamento


class Pix(Pagamento, Comprovante):
    DESCONTO = 0.05

    def calcular_total(self):
        return round(self._valor * (1 - Pix.DESCONTO), 2)

    def descricao(self):
        return "Pix (5% de desconto)"

    def gerar_comprovante(self):
        return f"Comprovante Pix: R$ {self.calcular_total():.2f} pagos à vista."
```

Teste classe PIX no main.py

```python
from models.pix import Pix
```

```python
print("--- Pagamento ---")

pix = Pix(3700)
cartao_3x = Cartao(3700, 3)
cartao_6x = Cartao(3700, 6)

print("RN8 - Pix (esperado 3515.0):", pix.calcular_total())
print("RN9 - Cartão em 3x (esperado 3700.0):", cartao_3x.calcular_total())
print("RN9 - Cartão em 6x (esperado 4070.0):", cartao_6x.calcular_total())

pagamentos = [pix, cartao_3x, cartao_6x]
for pagamento in pagamentos:
    print(pagamento.gerar_comprovante())
```

- **RN8:** Pix tem 5% de desconto.
- Pix não tem `__init__`: usa o construtor da classe mãe.
- `DESCONTO` é um atributo de classe: vale para todos os objetos Pix.
- `calcular_total()` e `gerar_comprovante()` são os métodos que os contratos exigem.
- Conta: 3700 × 0,95 = **3515**.

Fonte: src/models/pix.py

Cartao: super(), juros e parcelas (RN9 e RN10)

```
class Cartao(Pagamento, Comprovante):
```

```python
from models.comprovante import Comprovante
from models.pagamento import Pagamento


class Cartao(Pagamento, Comprovante):
    JUROS = 0.10

    def __init__(self, valor, parcelas=1):
        super().__init__(valor)
        if parcelas < 1 or parcelas > 12:
            raise ValueError("O cartão aceita de 1 a 12 parcelas.")
        self.__parcelas = parcelas

    @property
    def parcelas(self):
        return self.__parcelas

    def calcular_total(self):
        if self.__parcelas > 3:
            return round(self._valor * (1 + Cartao.JUROS), 2)
        return round(self._valor, 2)
```

Teste classe PIX no main.py

```python
from models.pix import Pix
```

```python
print("--- Pagamento ---")

pix = Pix(3700)
cartao_3x = Cartao(3700, 3)
cartao_6x = Cartao(3700, 6)

print("RN8 - Pix (esperado 3515.0):", pix.calcular_total())
print("RN9 - Cartão em 3x (esperado 3700.0):", cartao_3x.calcular_total())
print("RN9 - Cartão em 6x (esperado 4070.0):", cartao_6x.calcular_total())

pagamentos = [pix, cartao_3x, cartao_6x]
for pagamento in pagamentos:
    print(pagamento.gerar_comprovante())
```

- `super().__init__(valor)` chama o construtor de Pagamento.
- **RN10:** de 1 a 12 parcelas.
- **RN9:** acima de 3 parcelas, o total fica 10% maior.
- `parcelas=1` é opcional: `Cartao(3700)` vale 1 parcela.
- Em 6x: 3700 × 1,10 = **4070**.

 Fonte: PYTHON SOFTWARE FOUNDATION, Funções embutidas: super · src/models/cartao.py

Sobreposição e Sobercarga

Mesmo nome, comportamentos diferentes

Sobreposição (override)

A classe filha **redefine** um método da classe mãe, com o mesmo nome.

Pix e Cartao têm, cada um, o seu `calcular_total()`.

O Python usa a versão da classe do objeto.

No projeto: Pix dá desconto, Cartao cobra juros.

Sobrecarga (overload)

Mesmo nome de método com **parâmetros diferentes**.

O Python **não tem** sobrecarga nativa: a segunda definição apaga a primeira.

Simulamos com **parâmetro opcional**.

No projeto: `Cartao(3700)` e `Cartao(3700, 6)`.

Para lembrar: sobreposição acontece com herança entre classe pai e filho. Sobrecarga acontece dentro da mesma classe.

Fonte: KEOGH; GIANNINI, OOP desmistificado · PYTHON SOFTWARE FOUNDATION, O tutorial Python, seção Argumentos com valor padrão

Polimorfismo: a mesma chamada, respostas diferentes

Polimorfismo quer dizer “muitas formas”: o mesmo método funciona em objetos de classes diferentes, cada objeto executa essa ação de acordo com suas próprias características.

```python
print("--- Pagamento ---")

pix = Pix(3700)
cartao_3x = Cartao(3700, 3)
cartao_6x = Cartao(3700, 6)

print("RN8 - Pix (esperado 3515.0):", pix.calcular_total())
print("RN9 - Cartão em 3x (esperado 3700.0):", cartao_3x.calcular_total())
print("RN9 - Cartão em 6x (esperado 4070.0):", cartao_6x.calcular_total())

pagamentos = [pix, cartao_3x, cartao_6x]
for pagamento in pagamentos:
    print(pagamento.gerar_comprovante())
```

```python
class Pix(Pagamento, Comprovante):
    DESCONTO = 0.05

    def calcular_total(self):
        return round(self._valor * (1 - Pix.DESCONTO), 2)
```

```python
def calcular_total(self):
    if self.__parcelas > 3:
        return round(self._valor * (1 + Cartao.JUROS), 2)
    return round(self._valor, 2)
```

Pontos-chave

- O `for` percorre a lista e chama o método `calcular_total()` de cada objeto, sem precisar identificar se é Pix ou cartão.
- Cada forma de pagamento realiza o cálculo seguindo suas próprias regras.
- Funciona porque todos herdam de `Pagamento`.
- Uma forma nova de pagamento entra na lista sem mudar o `for`.

Fonte: PYTHON SOFTWARE FOUNDATION, O tutorial Python, seção Herança ·

Agregação: o pedido recebe itens que já existem

Composição · Carrinho e ItemCarrinho

O carrinho **cria** os seus itens.

O item não existe sem o carrinho.

No diagrama: losango **cheio**.

No código: `self.__itens.append(ItemCarrinho(produto, quantidade))`

Agregação · Pedido e ItemCarrinho

O pedido **recebe** itens que já existem.

O todo junta partes que foram criadas em outro lugar.

No diagrama: losango **vazio**.

No código: `self.__itens = cliente.carrinho.itens`

Quem cria a parte? Se é o todo, é composição. Se a parte chega pronta, é agregação.

Fonte: GUEDES, UML 2: uma abordagem prática · docs/diagrama-classes.md

Pedido: o construtor

O que acontece quando criamos um pedido?
O construtor prepara as informações, verifica o carrinho e configura o pagamento.

```
class Pedido:
```

```python
from models.cartao import Cartao
from models.pix import Pix


class Pedido:
    def __init__(self, numero, cliente, forma_pagamento, parcelas=1):
        if cliente.carrinho.esta_vazio():
            raise ValueError("O carrinho está vazio: não dá para fechar o pedido.")

        self.__numero = numero
        self.__cliente = cliente
        self.__itens = cliente.carrinho.itens
        self.__status = "aberto"

        if forma_pagamento == "pix":
            self.__pagamento = Pix(self.subtotal())
        elif forma_pagamento == "cartao":
            self.__pagamento = Cartao(self.subtotal(), parcelas)
        else:
            raise ValueError("Forma de pagamento inválida: use 'pix' ou 'cartao'.")
```

Teste classe PIX no main.py

```python
from models.pedido import Pedido
```

```python
print("--- Pedido ---")

pedido = Pedido(1, maria, "pix")
print("Total do pedido com Pix (esperado 3515.0):", pedido.total())

pedido.confirmar()
print(pedido)
print("RN11 - Estoque do notebook depois de confirmar (esperado 4):", notebook.estoque)
print("RN11 - Estoque do mouse depois de confirmar (esperado 18):", mouse.estoque)
print("RN7 - Carrinho depois do pedido (esperado Carrinho vazio.):", maria.carrinho)

pedido.cancelar()
print("RN11 - Estoque do notebook depois de cancelar (esperado 5):", notebook.estoque)
print("RN11 - Estoque do mouse depois de cancelar (esperado 20):", mouse.estoque)
```

- **RN6:** carrinho vazio não fecha pedido.
- **Agregação:** o pedido recebe os itens do carrinho.
- O pedido cria `Pix` ou `Cartao` a partir da escolha do cliente.
- O status começa em `"aberto"`.
- No arquivo do projeto, o parâmetro se chama `forma_pagamento`.

Pedido: total, confirmar e cancelar

```python
def total(self):
    return self.__pagamento.calcular_total()
```

```python
def confirmar(self):
    if self.__status != "aberto":
        raise ValueError(f"O pedido {self.__numero} já está {self.__status}.")

    for item in self.__itens:
        if not item.produto.tem_estoque(item.quantidade):
            raise ValueError(f"Estoque insuficiente de {item.produto.nome}.")
    for item in self.__itens:
        item.produto.baixar_estoque(item.quantidade)

    self.__status = "confirmado"
    self.__cliente.carrinho.limpar()
    self.__cliente.adicionar_pedido(self)
```

```python
def cancelar(self):
    if self.__status != "confirmado":
        raise ValueError("Só dá para cancelar um pedido confirmado.")
    for item in self.__itens:
        item.produto.repor_estoque(item.quantidade)
    self.__status = "cancelado"
```

- **Polimorfismo:** `total()` chama `calcular_total()` sem saber se é Pix ou Cartao.
- **RN11:** confirmar baixa o estoque; cancelar devolve.
- **RN7:** o carrinho esvazia depois do pedido.
- Status: aberto → confirmado → cancelado.

Fonte: src/models/pedido.py (trecho simplificado)

Pedido: __str__ e conferindo regras no main.py

```python
def __str__(self):
    linhas = [f"Pedido {self.__numero} ({self.__status}) - {self.__cliente.nome}"]
    for item in self.__itens:
        linhas.append(f"  {item}")
    linhas.append(f"  Subtotal: R$ {self.subtotal():.2f}")
    linhas.append(f"  Pagamento: {self.__pagamento.descricao()}")
    linhas.append(f"  TOTAL: R$ {self.total():.2f}")
    return "\n".join(linhas)
```

```python
print("--- Pedido ---")

pedido = Pedido(1, maria, "pix")
print("Total do pedido com Pix (esperado 3515.0):", pedido.total())

pedido.confirmar()
print(pedido)
print("RN11 - Estoque do notebook depois de confirmar (esperado 4):", notebook.estoque)
print("RN11 - Estoque do mouse depois de confirmar (esperado 18):", mouse.estoque)
print("RN7 - Carrinho depois do pedido (esperado Carrinho vazio.):", maria.carrinho)

pedido.cancelar()
print("RN11 - Estoque do notebook depois de cancelar (esperado 5):", notebook.estoque)
print("RN11 - Estoque do mouse depois de cancelar (esperado 20):", mouse.estoque)
```

- **Polimorfismo:** `total()` chama `calcular_total()` sem saber se é Pix ou Cartao.
- **RN11:** confirmar baixa o estoque; cancelar devolve.
- **RN7:** o carrinho esvazia depois do pedido.
- Status: aberto → confirmado → cancelado.

No main.py

- Cada `print` mostra o valor **esperado** e o valor **calculado**.
- Se os dois são iguais, a regra está funcionando.
- **Teste de erro:** tire o `#` de uma linha, rode e leia o `ValueError`. Depois coloque o `#` de volta.
- Para rodar: `py src/main.py`

Fonte: src/models/pedido.py (trecho simplificado)

A compra de teste: confira os valores no seu main.py

R$ 3.700,00

Total do carrinho

1 Notebook (3500) + 2 Mouses (100 cada)

R$ 3.515,00

Pagando com Pix

RN8: 5% de desconto. 3700 × 0,95

R$ 4.070,00

Cartão em 6 vezes

RN9: acima de 3x, 10% a mais. 3700 × 1,10

No programa os valores aparecem como 3700.0, 3515.0 e 4070.0. Depois de confirmar o pedido, o estoque do notebook cai de 5 para 4.

Fonte: src/main.py · RN8 e RN9

Conceitos associados - Pagamento, Pix, Cartao e Pedido

1 · Herança

Uma classe filha recebe atributos e métodos da classe mãe. Relação “é um”.

No projeto: `Pix(Pagamento)` e `Cartao(Pagamento)`.

2 · Classe abstrata

Modelo que não cria objetos e obriga as filhas a escrever certos métodos.

No projeto: `Pagamento(ABC)` com `calcular_total()` abstrato.

3 · Interface

Contrato só com métodos, sem atributos e sem código pronto.

No projeto: `Comprovante` com `gerar_comprovante()`.

4 · Sobreposição

A filha redefine um método da mãe com o mesmo nome.

No projeto: cada pagamento tem o seu `calcular_total()`.

5 · Polimorfismo

A mesma chamada funciona em objetos de classes diferentes.

No projeto: a lista de pagamentos e `Pedido.total()`.

6 · Agregação

O todo recebe partes que já existem.

No projeto: o Pedido recebe os itens do carrinho.